

Федеральное государственное автономное образовательное учреждение высшего  
образования



**«Московский государственный технический университет  
имени Н.Э. Баумана»**

**(национальный исследовательский университет)**

**(МГТУ им. Н.Э. Баумана)**

---

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ  
КАФЕДРА КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)

## **О т ч е т**

**по лабораторной работе № 5**

**Название лабораторной работы: 5. С#. Коллекции. Создание  
приложений с графическим интерфейсом**

**Дисциплина: Алгоритмизация и программирование**

Студент гр. ИУ6-23Б \_\_\_\_\_ **С.М. Соболев**  
(Подпись, дата) (И.О. Фамилия)

Преподаватель \_\_\_\_\_ **О.А. Веселовская**  
(Подпись, дата) (И.О. Фамилия)

Москва, 2026

Цель работы: разработать оконное приложение на C# для шифрования и дешифровки текста на основе циклического цифрового ключа. Освоить практическое применение коллекций для хранения истории операций пользователя. Закрепить навыки проектирования графических интерфейсов, объектно-ориентированного программирования и отладки кода.

Задание: Разработать приложения, выполняющие любое задание лабораторной работы 4, с использованием коллекций и графическим интерфейсом, выполнить тестирование и отладку. В отчете обосновать выбор коллекции, привести диаграмму классов, текст программы и результаты тестирования. Дана последовательность строк. Строки содержат слова, разделенные пробелом. Используя цифровой шифр, например 31206, зашифровать каждую строку по следующей методике: 31206 312063 12 063

Пирог сгорел до тла

Ткток фкррло ер есг , то есть, к каждой букве применяют соответствующую цифру для определения смещения этой буквы, с целью получения буквы шифра.

Написать программу, обеспечивающую ввод строк, шифровку и дешифровку. Вывести на экран зашифрованную и подвергнутую дешифровке последовательности строк.

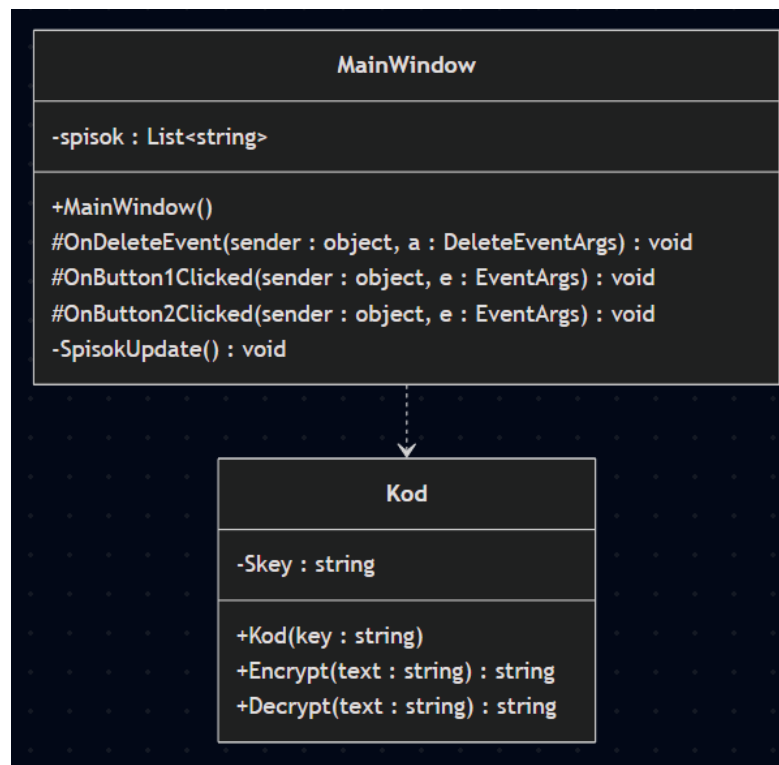


Рисунок 1 – Диаграмма классов

```

using System;
using Gtk;
using System.Collections.Generic;

public partial class MainWindow : Gtk.Window
{
    private List<string> spisok;
    public MainWindow() : base(Gtk.WindowType.Toplevel)
    {
        Build();
        spisok = new List<string>();
    }

    protected void OnDeleteEvent(object sender, DeleteEventArgs a)
    {
        Application.Quit();
        a.RetVal = true;
    }

    protected void OnButton1Clicked(object sender, EventArgs e)
    {
        string text = entry1.Text;
        Kod kod = new Kod(entry3.Text);
        entry1.Text = kod.Encrypt(text);
        spisok.Add("Операция: Шифрование. Текст: " + text);
        SpisokUpdate();
    }

    protected void OnButton2Clicked(object sender, EventArgs e)
    {
        string text = entry2.Text;
        Kod kod = new Kod(entry3.Text);
        entry2.Text = kod.Decrypt(text);
        spisok.Add("Операция: Дешифрование. Текст: " + text);
        SpisokUpdate();
    }

    private void SpisokUpdate()
    {
        textview1.Buffer.Text = "";
        foreach (string el in spisok)
        {
            Gtk.TextIter iter = textview1.Buffer.EndIter;
            textview1.Buffer.Insert(ref iter, el + '\n');
        }
    }
}

```

Рисунок 2 – Код приложения

```

class Kod
{
    private string Skey;
    public Kod(string key)
    {
        Skey = key;
    }

    public string Encrypt(string text)
    {
        string result = "";
        int keyIndex = 0;
        for (int i = 0; i < text.Length; ++i)
        {
            if (char.IsLetter(text[i]))
            {
                bool caps = char.IsUpper(text[i]);
                char baseChar = caps ? 'A' : 'a';
                result += (char)((((text[i] + Skey[keyIndex]) - '0' - baseChar) % 32) + baseChar);
                keyIndex = (++keyIndex) % Skey.Length;
            }
            else
            {
                result += text[i];
            }
        }
        return result;
    }

    public string Decrypt(string text)
    {
        string result = "";
        int keyIndex = 0;
        for (int i = 0; i < text.Length; ++i)
        {
            if (char.IsLetter(text[i]))
            {
                bool caps = char.IsUpper(text[i]);
                char baseChar = caps ? 'A' : 'a';
                result += (char)((((text[i] - Skey[keyIndex]) + '0' + baseChar) % 32) + baseChar);
                keyIndex = (++keyIndex) % Skey.Length;
            }
            else
            {
                result += text[i];
            }
        }
        return result;
    }
}

```

Рисунок 3 – Код приложения

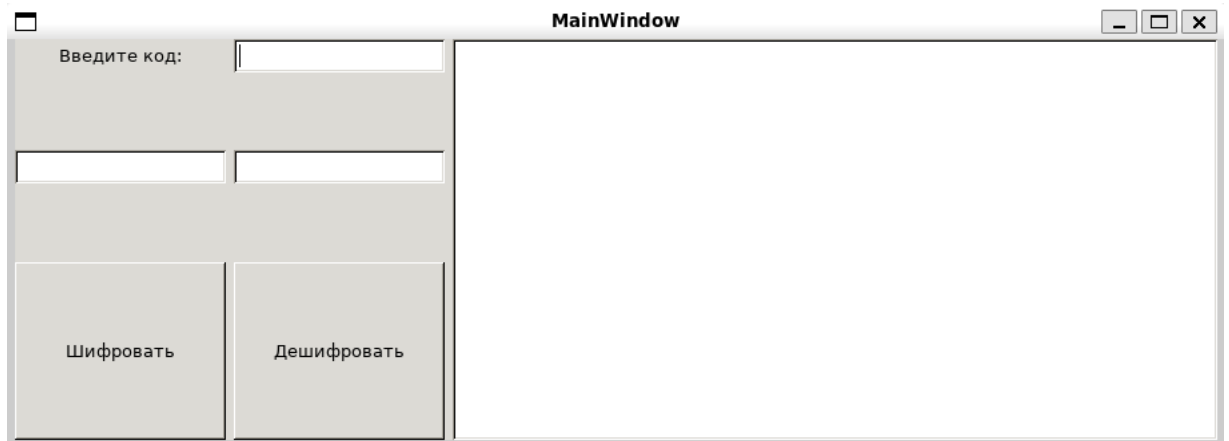


Рисунок 4 – Интерфейс приложения

Обоснование выбора коллекции. Для хранения истории операций выбран List, так как он обладает динамическим размером и автоматически расширяется при добавлении новых записей. Коллекция обеспечивает строгую типизацию данных и высокую производительность при последовательном добавлении логов и их отображении через цикл.

Вывод: в ходе работы успешно создано приложение с графическим интерфейсом, реализующее заданный алгоритм шифрования строк. Применение коллекции List позволило эффективно организовать сохранение и динамический вывод истории действий на экран. Результаты тестирования подтверждают полную работоспособность алгоритмов преобразования текста и корректное обновление элементов интерфейса.